

Data Visualization

- Most people visualize information better when they see it in graphic, versus textual, format. Graphics help people see relationships and make comparisons with greater ease.
- Even if you can deal with the abstraction of textual data with ease, performing data analysis is all about communication. Unless you can communicate your ideas to other people, the act of obtaining, shaping, and analyzing the data has little value beyond your own personal needs.
- Fortunately, Python makes the task of converting your textual data into graphics relatively easy using **MatPlotLib**, which is actually a simulation of the MATLAB application

Matplotlib: Visualization with Python

- Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python.
- Using Matplotlib one can create variety of plots like:
 - Bar Graph
 - Pie Chart
 - Box Plot
 - Histogram
 - Line Chart and Subplots
 - Scatter Plot
- Basic plot customizations, with a focus on plot legends and text, titles, axes labels and plot layout.
- Saving, showing, clearing, ... your plots: show the plot, save one or more figures to, for example, pdf files, clear the axes, clear the figure or close the plot, etc.
- Matplotlib can be used in Python scripts, the Python and IPython shells, the Jupyter notebook, web application servers, and graphical user interface toolkits.

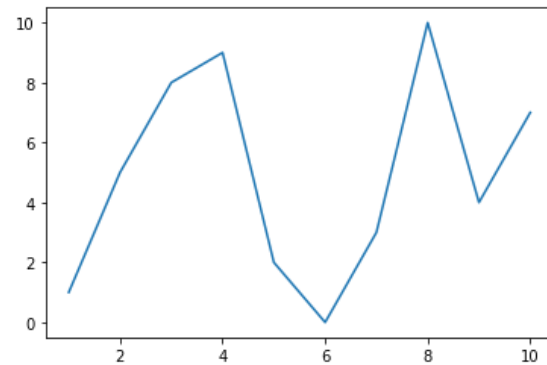
Starting with a Graph

- A graph or chart is simply a visual representation of numeric data. MatPlotLib makes a large number of graph and chart types available to you. Of course, you can choose any of the common graph and graph types such as bar charts, line graphs, or pie charts.
- As with MATLAB, you also have access to a huge number of statistical plot types, such as box plots, error bar charts, and histograms.

Defining the plot

- import the pyplot module of the matplotlib library under the alias plt to define various types of plots:

```
In [5]: values = [1, 5, 8, 9, 2, 0, 3, 10, 4, 7]
import matplotlib.pyplot as plt
plt.plot(range(1,11), values)
plt.show()
```



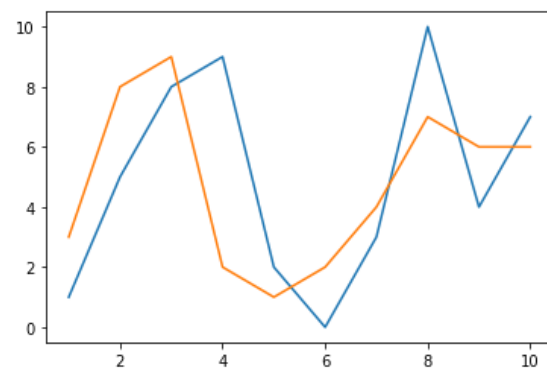
In this case, the code tells the `plt.plot()` function to create a plot using x-axis values between 1 and 11 and y-axis values as they appear in `values`. Calling `plt.show()` displays the plot in a separate dialog box

Drawing multiple lines and plots

To create multiple line plots using Matplotlib, you simply call `plt.plot()` multiple times — once for each plot line, as shown in the following example.

```
In [5]: values = [1, 5, 8, 9, 2, 0, 3, 10, 4, 7]
values2 = [3, 8, 9, 2, 1, 2, 4, 7, 6, 6]
import numpy as np

import matplotlib.pyplot as plt
plt.plot(range(1,11), values)
plt.plot(range(1,11), values2)
plt.show()
```



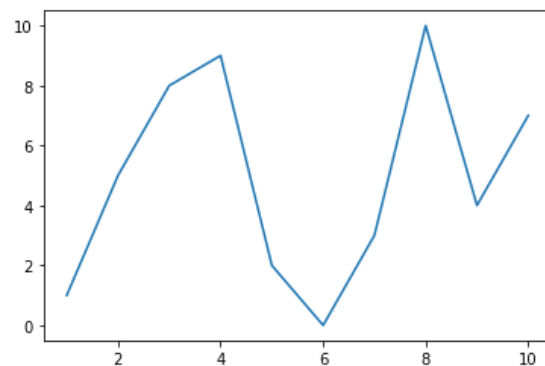
```
In [13]: values = [1, 5, 8, 9, 2, 0, 3, 10, 4, 7]
values2 = [3, 8, 9, 2, 1, 2, 4, 7, 6, 6]
print(zip(values, values2))
import pandas as pd
df=pd.DataFrame(list(zip(values, values2)))
print(df)
```

```
<zip object at 0x00000FFE1C5AEC8>
  0  1
0  1  3
1  5  8
2  8  9
3  9  2
4  2  1
5  0  2
6  3  4
7 10  7
8  4  6
9  7  6
```

Saving your work

Often you need to save a copy of your work to disk for later reference or to use as part of a larger report. The easiest way to accomplish this task is to click Save the Figure. You see a dialog box that you can use to save the figure to disk. However, you sometimes need to save the graphic automatically rather than wait for the user to do it. In this case, you can save it programmatically using the `plt.savefig()` function, as shown in the following code:

```
In [7]: values = [1, 5, 8, 9, 2, 0, 3, 10, 4, 7]
import matplotlib.pyplot as plt
plt.plot(range(1,11), values)
plt.savefig('MySamplePlot.png', format='png')
```



In this case, you must provide a minimum of two inputs. The first input is the filename. You may optionally include a path for saving the file. The second input is the file format. In this case, the example saves the file in Portable Network Graphic (PNG) format, but you have other options: Portable Document Format (PDF), Postscript (PS), Encapsulated Postscript (EPS), and Scalable Vector Graphics (SVG).

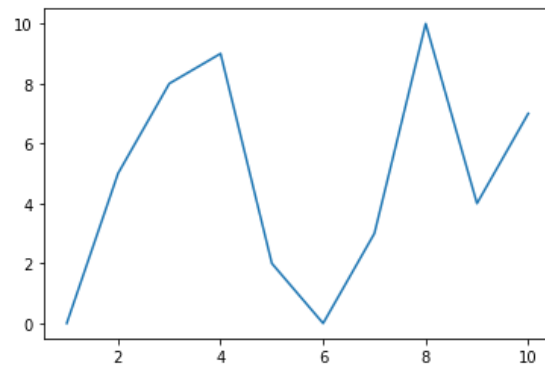
Setting the Axis, Ticks, Grids

It's hard to know what the data actually means unless you provide a unit of measure or at least some means of performing comparisons. The use of axes, ticks, and grids make it possible to illustrate graphically the relative size of data elements so that the viewer gains proper comparatives.

Getting the axes

The axes define the x and y plane of the graphic. The x axis runs horizontally, and the y axis runs vertically. In many cases, you can allow Matplotlib to perform any required formatting for you. However, sometimes you need to obtain access to the axes and format them manually. The following code shows how to obtain access to the axes for a plot:

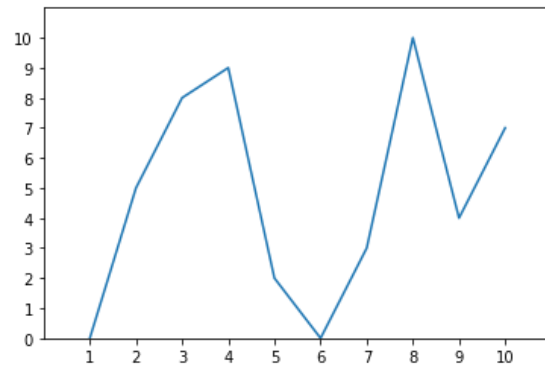
```
In [2]: values = [0, 5, 8, 9, 2, 0, 3, 10, 4, 7]
import matplotlib.pyplot as plt
ax = plt.axes()
plt.plot(range(1,11), values)
plt.show()
```



In this case, you simply turn on the default axes by calling `plt.axes()`; then you place a handle to the axes in `ax`

Formatting the axes

```
In [9]: values = [0, 5, 8, 9, 2, 0, 3, 10, 4, 7]
import matplotlib.pyplot as plt
ax = plt.axes()
ax.set_xlim([0, 11])
ax.set_ylim([0, 11])
ax.set_xticks([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
ax.set_yticks([0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
plt.plot(range(1,11), values)
plt.show()
```

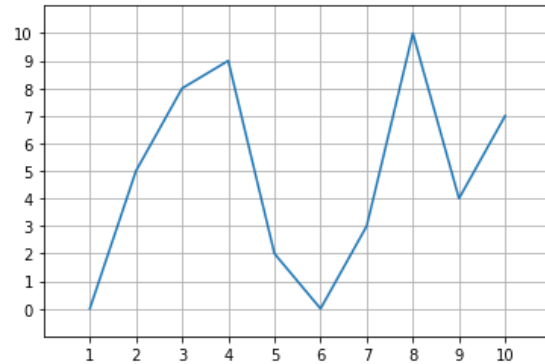


- In this case, the `set_xlim()` and `set_ylim()` calls change the axes limits — the length of each axis.
- The `set_xticks()` and `set_yticks()` calls change the ticks used to display data.
- The ways in which you can change a graph using these calls can become quite detailed.

Adding grids

- Grid lines make it possible to see the precise value of each element of a graph. You can more quickly determine both the x and y coordinate, which allow you to perform comparisons of individual points with greater ease.

```
In [10]: values = [0, 5, 8, 9, 2, 0, 3, 10, 4, 7]
import matplotlib.pyplot as plt
ax = plt.axes()
ax.set_xlim([0, 11])
ax.set_ylim([-1, 11])
ax.set_xticks([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
ax.set_yticks([0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
ax.grid()
plt.plot(range(1,11), values)
plt.show()
```



Defining the Line Appearance

- Just drawing lines on a page won't do much for you if you need to help the viewer understand the importance of your data. In most cases, you need to use different line styles to ensure that the viewer can tell one data grouping from another.
- However, to emphasize the importance or value of a particular data grouping, you need to employ color. The use of color communicates all sorts of ideas to the viewer. For example, green often denotes that something is safe, while red communicates danger.
- The following sections help you understand how to work with line style and color to communicate ideas and concepts to the viewer without using any text.

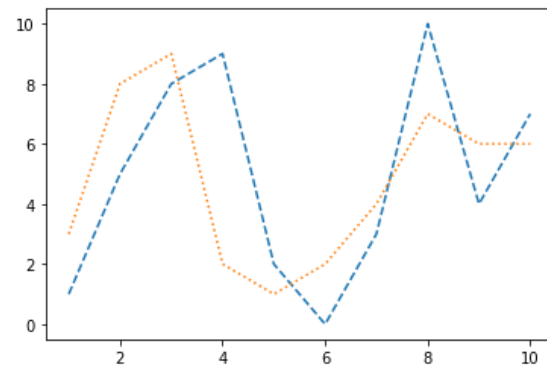
Working with line styles

- Line styles help differentiate graphs by drawing the lines in various ways. Following figure shows different character and according line style:

line styles



```
In [11]: values = [1, 5, 8, 9, 2, 0, 3, 10, 4, 7]
          values2 = [3, 8, 9, 2, 1, 2, 4, 7, 6, 6]
          import matplotlib.pyplot as plt
          plt.plot(range(1,11), values, '--')
          plt.plot(range(1,11), values2, ':')
          plt.show()
```

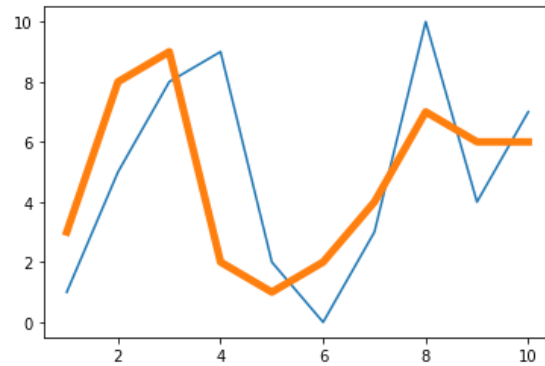


Line Width

- Matplotlib allows you to adjust the line width of a graph plot using the linewidth attribute. By default, linewidth=1

- If you want to make the line width of a graph plot thinner, then you can make linewidth less than 1, such as 0.5 or 0.25.
- If you want to make the line width of the graph plot thicker, then you can make linewidth greater than 1.

```
In [12]: values = [1, 5, 8, 9, 2, 0, 3, 10, 4, 7]
values2 = [3, 8, 9, 2, 1, 2, 4, 7, 6, 6]
import matplotlib.pyplot as plt
plt.plot(range(1,11), values, '-')
plt.plot(range(1,11), values2, '-',linewidth=5)
plt.show()
```

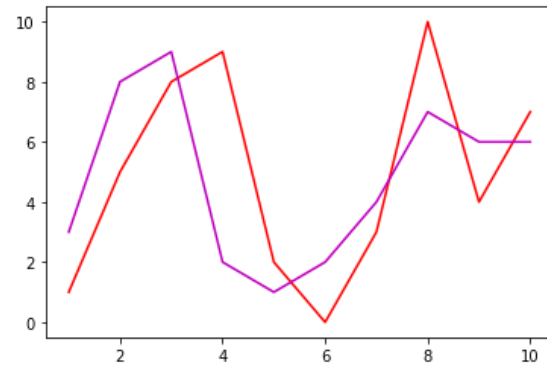


Using colors

Matplotlib has an additional parameter to control the colour and style of the plot.

Table 9-2	MatPlotLib Colors
Character	Color
'b'	Blue
'g'	Green
'r'	Red
'c'	Cyan
'm'	Magenta
'y'	Yellow
'k'	Black
'w'	White

```
In [13]: values = [1, 5, 8, 9, 2, 0, 3, 10, 4, 7]
values2 = [3, 8, 9, 2, 1, 2, 4, 7, 6, 6]
import matplotlib.pyplot as plt
plt.plot(range(1,11), values, 'r')
plt.plot(range(1,11), values2, 'm')
plt.show()
```



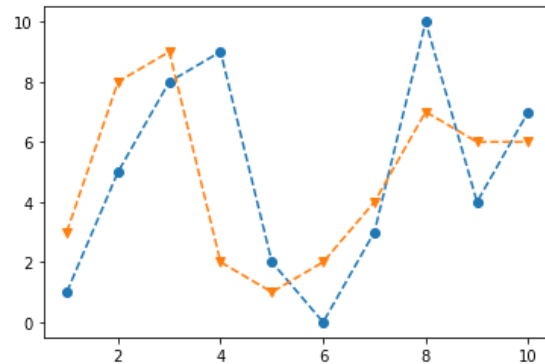
Adding markers

- Markers add a special symbol to each data point in a line graph. Unlike line style and color, markers tend to be a little less susceptible to accessibility and printing issues. Even when the specific marker isn't clear, people can usually differentiate one marker from the other.

Table 9-3 Matplotlib Markers	
<i>Character</i>	<i>Marker Type</i>
'.'	Point
','	Pixel
'o'	Circle
'v'	Triangle 1 down
'^'	Triangle 1 up
'<'	Triangle 1 left
'>'	Triangle 1 right
'1'	Triangle 2 down
'2'	Triangle 2 up
'3'	Triangle 2 left
'4'	Triangle 2 right
's'	Square
'p'	Pentagon
'*'	Star
'h'	Hexagon style 1
'H'	Hexagon style 2
'+'	Plus
'x'	X
'D'	Diamond
'd'	Thin diamond
' '	Vertical line
'_'	Horizontal line

As with line style and color, you add markers as the third argument to a `plot()` call. In the following example, you see the effects of combining line style with a marker to provide a unique line graph presentation.

```
In [14]: values = [1, 5, 8, 9, 2, 0, 3, 10, 4, 7]
values2 = [3, 8, 9, 2, 1, 2, 4, 7, 6, 6]
import matplotlib.pyplot as plt
plt.plot(range(1,11), values, 'o--')
plt.plot(range(1,11), values2, 'v--')
plt.show()
```

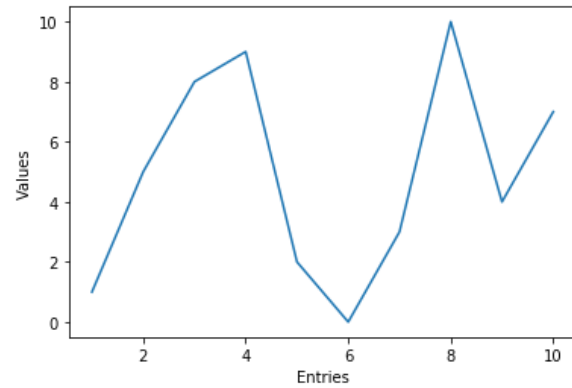


Using Labels, Annotations, and Legends

- To fully document your graph, you usually have to resort to labels, annotations, and legends.
- **Label:** Provides positive identification of a particular data element or grouping. The purpose is to make it easy for the viewer to know the name or kind of data illustrated.
- **Annotation:** Augments the information the viewer can immediately see about the data with notes, sources, or other useful information. In contrast to a label, the purpose of annotation is to help extend the viewer's knowledge of the data rather than simply identify it.
- **Legend:** Presents a listing of the data groups within the graph and often provides cues (such as line type or color) to make identification of the data group easier.

Adding labels

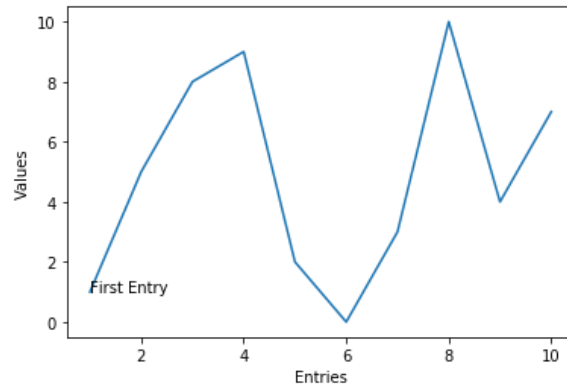
```
In [15]: values = [1, 5, 8, 9, 2, 0, 3, 10, 4, 7]
import matplotlib.pyplot as plt
plt.xlabel('Entries')
plt.ylabel('Values')
plt.plot(range(1,11), values)
plt.show()
```



Annotating the chart

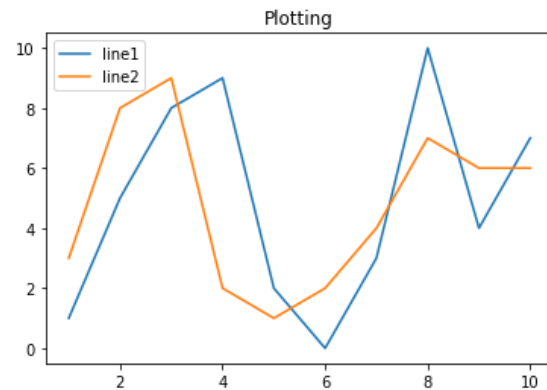
- You use annotation to draw special attention to points of interest on a graph. For example, you may want to point out that a specific data point is outside the usual range expected for a particular dataset.

```
In [16]: values = [1, 5, 8, 9, 2, 0, 3, 10, 4, 7]
import matplotlib.pyplot as plt
plt.xlabel('Entries')
plt.ylabel('Values')
plt.annotate(xy=[1,1], s='First Entry')
plt.plot(range(1,11), values)
plt.show()
```



Creating a legend and title to graph

```
In [17]: values = [1, 5, 8, 9, 2, 0, 3, 10, 4, 7]
values2 = [3, 8, 9, 2, 1, 2, 4, 7, 6, 6]
import matplotlib.pyplot as plt
line1 = plt.plot(range(1,11), values, label='line1')
line2 = plt.plot(range(1,11), values2, label='line2')
plt.legend(loc='upper left')
plt.title('Plotting')
plt.show()
```



- Creating a graphic presentation of your data is essential if you want to help other people understand what you're trying to say.

- Even though you can see what the numbers mean in your mind
- No book on data science would be complete without exploring scatterplots, which are used to help people see patterns in seemingly unrelated data points.
- You also get to work with both directed and undirected graphs, which is fine for social media analysis.

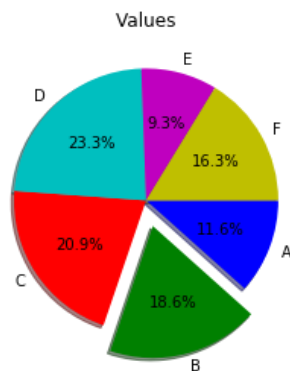
Choosing the Right Graph

- The kind of graph you choose determines how people view the associated data, so choosing the right graph from the outset is important.
- For example, if you want to show how various data elements contribute toward a whole, you really need to use a pie chart. On the other hand, when you want people to form opinions on how data elements compare, you use a bar chart.
- The idea is to choose a graph that naturally leads people to draw the conclusion
- The following sections describe the various graph types and provide you with basic examples of how to use them.

Showing parts of a whole with pie charts

- **Pie charts focus on showing parts of a whole.** The entire pie would be 100 percent. The question is how much of that percentage each value occupies. The following example shows how to create a pie chart with many of the special features in place:
- Example---

```
In [18]: import matplotlib.pyplot as plt
values = [5, 8, 9, 10, 4, 7]
colors = ['b', 'g', 'r', 'c', 'm', 'y']
labels = ['A', 'B', 'C', 'D', 'E', 'F']
explode = (0, 0.2, 0, 0, 0, 0)
plt.pie(values, colors=colors, labels=labels,
        explode=explode, autopct='%1.1f%%',
        counterclock=False, shadow=True)
plt.title('Values')
plt.show()
```

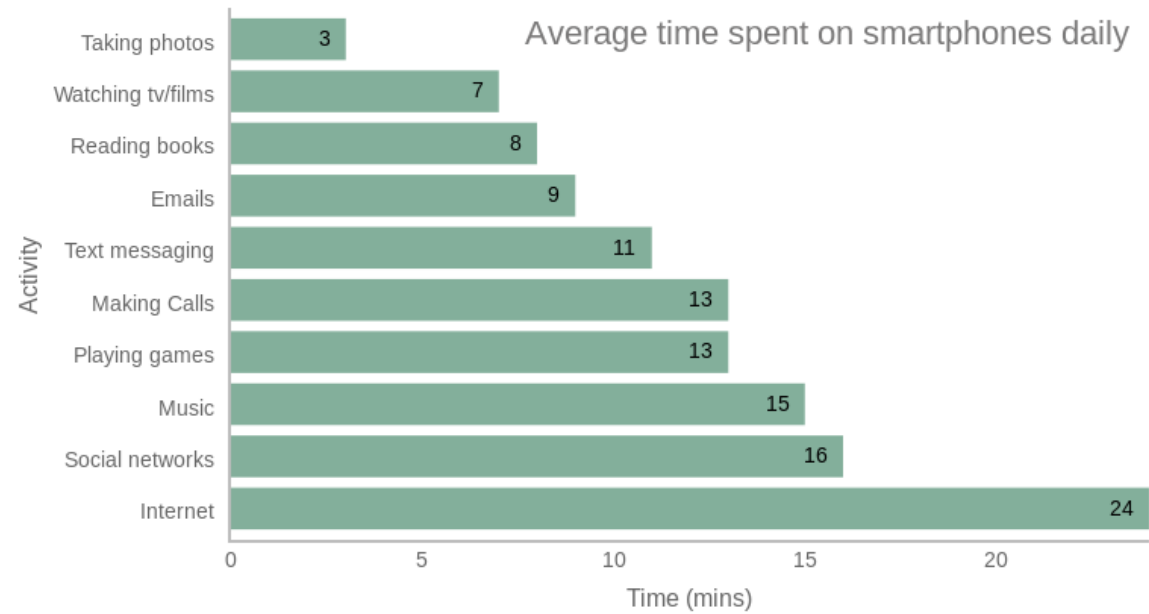


- You could create a basic pie chart using just the values as input. The colors parameter lets you choose custom colors for each pie wedge. You use the labels parameter to identify each wedge. In many cases, you need to make one wedge stand out from the others, so you add the explode parameter with list of explode values.

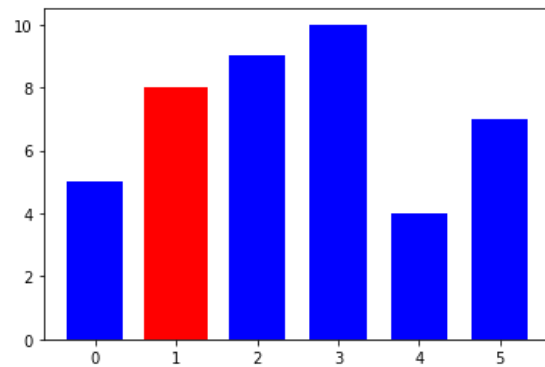
- A value of 0 keeps the wedge in place — any other value moves the wedge out from the center of the pie.

Creating comparisons with bar charts

- Bar charts make comparing values easy. The wide bars and segregated measurements emphasize the differences between values, rather than the flow of one value to another as a line graph would do.
- These charts are useful for displaying data that is classified into nominal or ordinal categories.



```
In [19]: import matplotlib.pyplot as plt
values = [5, 8, 9, 10, 4, 7]
widths = [0.7, 0.8, 0.7, 0.7, 0.7, 0.7]
colors = ['b', 'r', 'b', 'b', 'b', 'b']
plt.bar(range(0, 6), values, width=widths,
        color=colors, align='center')
plt.show()
```



Example of Bar Chart with Multiple X's

- To include multiple X values on the same chart, we can reduce the width of the bars and then place the indices one bar's width further from the y axis.

```
In [20]: import numpy as np

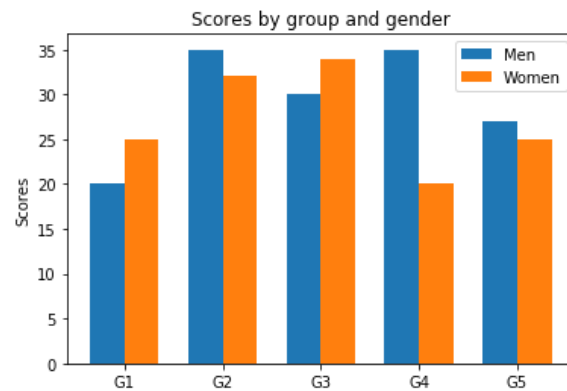
N = 5
men_means = (20, 35, 30, 35, 27)
women_means = (25, 32, 34, 20, 25)

ind = np.arange(N)
print(ind)
width = 0.35
plt.bar(ind, men_means, width, label='Men')
plt.bar(ind + width, women_means, width, label='Women')

plt.ylabel('Scores')
plt.title('Scores by group and gender')

plt.xticks(ind + width / 2, ('G1', 'G2', 'G3', 'G4', 'G5'))
plt.legend(loc='best')
plt.show()
```

[0 1 2 3 4]



Stacked Bar Chart

With stacked bar charts we need to provide the parameter `bottom`, this informs matplotlib where the bar should start from, so we will add up the values below.

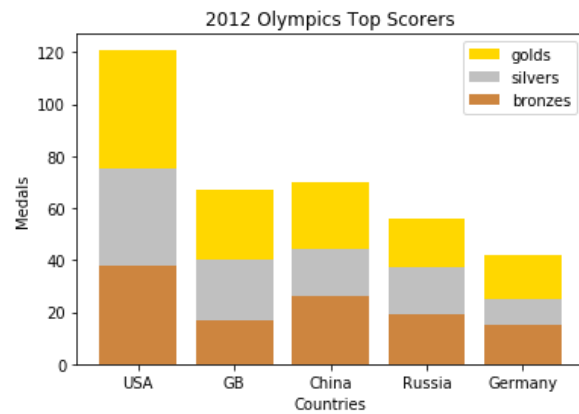
- `bottom` : scalar or array-like, optional The y coordinate(s) of the bars bases (default: 0).

```
In [21]: countries = ['USA', 'GB', 'China', 'Russia', 'Germany']
bronzes = np.array([38, 17, 26, 19, 15])
silvers = np.array([37, 23, 18, 18, 10])
golds = np.array([46, 27, 26, 19, 17])
ind = [x for x, _ in enumerate(countries)]
print(ind)
plt.bar(ind, golds, width=0.8, label='golds', color='gold', bottom=silvers+bronzes)
plt.bar(ind, silvers, width=0.8, label='silvers', color='silver', bottom=bronzes)
plt.bar(ind, bronzes, width=0.8, label='bronzes', color='#CD853F')

plt.xticks(ind, countries)
plt.ylabel("Medals")
plt.xlabel("Countries")
plt.legend(loc="upper right")
plt.title("2012 Olympics Top Scorers")

plt.show()
```

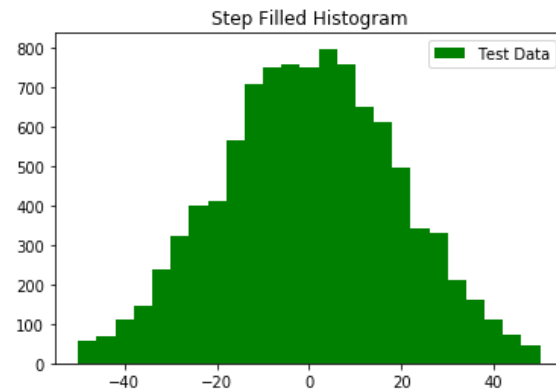
```
[0, 1, 2, 3, 4]
```



Showing distributions using histograms

- Histograms categorize data by breaking it into bins, where each bin contains a subset of the data range. A histogram then displays the number of items in each bin so that you can see the distribution of data and the progression of data from bin to bin. In most cases, you see a curve of some type, such as a bell curve. The following example shows how to create a histogram with randomized data:

```
In [57]: import numpy as np
import matplotlib.pyplot as plt
x = 20 * np.random.randn(10000)
plt.hist(x, 25, range=(-50,50), histtype='stepfilled', align='mid', color='g', label='Test Data')
plt.legend()
plt.title('Step Filled Histogram')
plt.show()
```

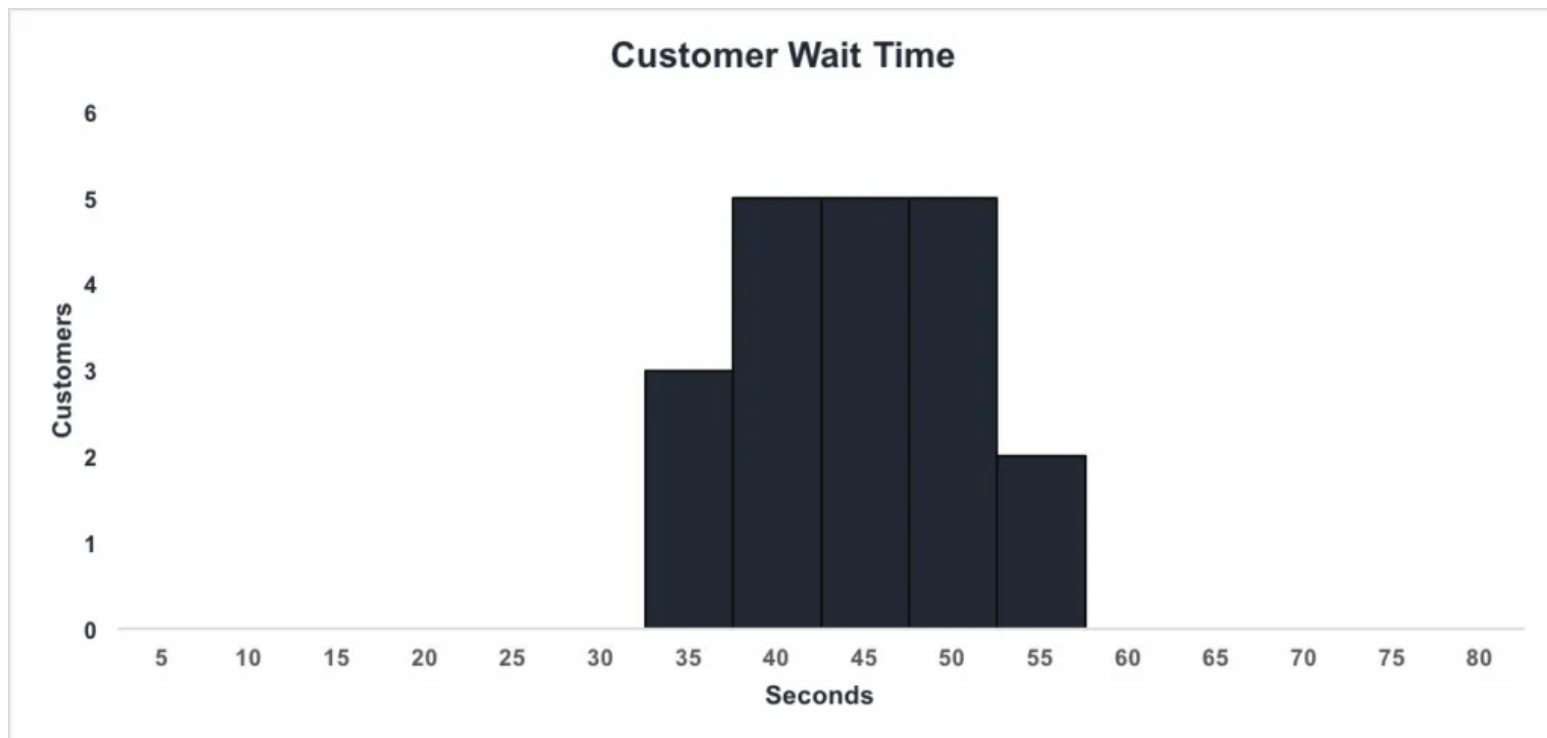


- In this case, the input values are a series of random numbers. The distribution of these numbers should show a type of bell curve. As a minimum, you must provide a series of values, x in this case, to plot. The second argument contains the number of bins to use when creating the data intervals.
- The default value is 10. Using the range parameter helps you focus the histogram on the relevant data and exclude any outliers.
- **range** The lower and upper range of the bins.
- **hystype**: The type of histogram to draw. Default is 'bar'
 - 'bar' is a traditional bar-type histogram. If multiple data are given the bars are arranged side by side.
 - 'barstacked' is a bar-type histogram where multiple data are stacked on top of each other.
 - 'step' generates a lineplot that is by default unfilled.
 - 'stepfilled' generates a lineplot that is by default filled.
- **align** parameter determines the alignment of each bar along the baseline. Use the color parameter to control the colors of the bars.
- **label** parameter doesn't actually appear unless you also create a legend
- Random data varies call by call. Every time you run the example, you see slightly different results because the random-generation process differs.

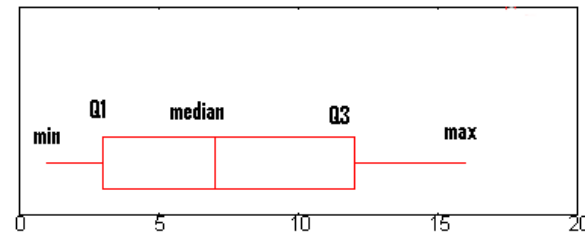
Example:Depicting general waiting time of 20 customers at bank

- We can see that There are 3 customers waiting between 1 and 35 seconds

Customer Wait Time in Seconds (n=20)	
43.1	42.2
35.6	45.5
37.6	30.3
36.5	31.4
45.3	35.6
43.5	45.2
40.3	54.1
50.2	45.6
47.3	36.5
31.2	43.1



Depicting groups using box plots



- **The box and whiskers chart shows you how your data is spread out.**
- The minimum (the smallest number in the data set). The minimum is shown at the far left of the chart, at the end of the left “whisker.”
- First quartile, Q1, is the far left of the box (or the far right of the left whisker).
- The median is shown as a line in the center of the box.
- Third quartile, Q3, shown at the far right of the box (at the far left of the right whisker).
- The maximum (the largest number in the data set), shown at the far right of the box.
- **Data sets can sometimes contain outliers that are suspected to be anomalies. Small circles or unfilled dots are drawn on the chart to indicate where suspected outliers lie. Filled circles are used for known outliers.**
- A box and whisker plot can provide additional detail while allowing multiple sets of data to be displayed in the same graph.
- The spacing shown within a box plot helps indicate the skew and dispersion of the data.

How to Read a Box Plot: Steps

Step 1: Find the minimum. The minimum is the far left hand side of the graph, at the tip of the left whisker. For this graph, the left whisker end is at approximately 0.75.

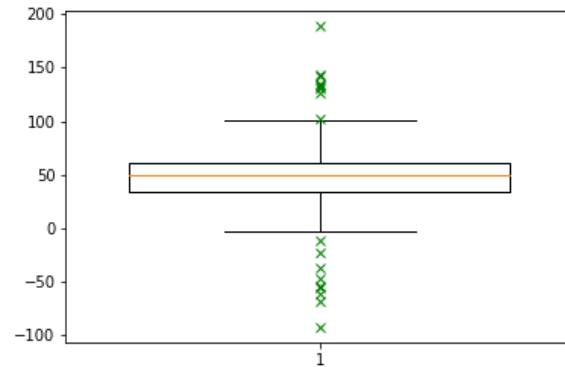
Step 2: Find Q1, the first quartile. Q1 is represented by the far left hand side of the box. In this case, about 2.5.

Step 3: Find the median. The median is represented by the vertical bar. In this boxplot, it can be found at about 6.5.

Step 4: Find Q3, the third quartile. Q3 is the far right hand edge of the box, at about 12 in this graph.

Step 5: Find the maximum. The maximum is the end of the “whiskers”: in this graph, at approximately 16.

```
In [59]: import numpy as np
import matplotlib.pyplot as plt
spread = 100 * np.random.rand(100)
center = np.ones(50) * 50
flier_high = 100 * np.random.rand(10) + 100
flier_low = -100 * np.random.rand(10)
data = np.concatenate((spread, center, flier_high, flier_low))
plt.boxplot(data, sym='gx', widths=.75, notch=False)
plt.show()
```

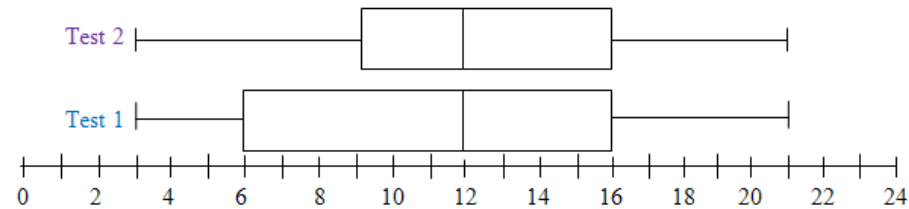


Here are how these techniques work:

- spread: Contains a set of random numbers between 0 and 100
 - center: Provides 50 values directly in the center of the range of 50
 - flier_high: Simulates outliers between 100 and 200
 - flier_low: Simulates outliers between 0 and -100
-
- The code combines all these values into a single dataset using concatenate().
 - The call to boxplot() requires only data as input. All other parameters have default settings. In this case, the code sets the presentation of outliers to green Xs by setting the sym parameter.
 - You use widths to modify the size of the box (made extra large in this case to make the box easier to see).
 - Finally, you can create a square box or a box with a notch using the notch
-
- The box shows the three data points as the box, with the red line in the middle being the median. The two black horizontal lines connected to the box by whiskers show the upper and lower limits (for four quartiles).
 - The outliers appear above and below the upper and lower limit lines as green Xs.

Example

1. **box-and-whisker plots represent quiz scores out of 25 points for Quiz 1 and Quiz 2 for the same class.**

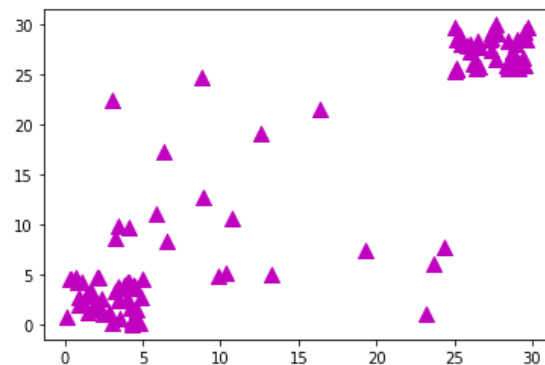


These box-and-whisker plots show that the lowest score, highest score, and Q3 are all the same for both exams, so performance on the two exams were quite similar. However, the movement Q1 up from a score of 6 to a score of 9 indicates that there was an overall improvement. On the first test, approximately 75% of the students scored at or above a score of 6. On the second test, the same number of students (75%) scored at or above a score of 9.

Seeing data patterns using scatterplots

Scatterplots show clusters of data rather than trends (as with line graphs) or discrete values (as with bar charts). The purpose of a scatterplot is to help you see data patterns. The following example shows how to create a scatterplot using randomized data:

```
In [24]: import numpy as np
import matplotlib.pyplot as plt
x1 = 5 * np.random.rand(40)
x2 = 5 * np.random.rand(40) + 25
x3 = 25 * np.random.rand(20)
x = np.concatenate((x1, x2, x3))
y1 = 5 * np.random.rand(40)
y2 = 5 * np.random.rand(40) + 25
y3 = 25 * np.random.rand(20)
y = np.concatenate((y1, y2, y3))
plt.scatter(x, y, s=[100], marker='^', c='m')
plt.show()
```



In this case, the `s` parameter determines the size of each data point. The `marker` parameter determines the data point shape. You use the `c` parameter to define the colors for all the data points, or you

can define a separate color for individual data points.

Creating Advanced Scatterplots

Scatterplots are especially important for data science because they can show data patterns that aren't obvious when viewed in other ways. You can see data groupings with relative ease and help the viewer understand when data belongs to a particular group. You can also show overlaps between groups and even demonstrate when certain data is outside the expected range. Showing these various kinds of relationships in the data is an advanced technique that you need to know in order to make the best use of Matplotlib.

Depicting groups

Color is the third axis when working with a scatterplot. Using color lets you highlight groups so that others can see them with greater ease. The following example shows how you can use color to show groups within a scatterplot:

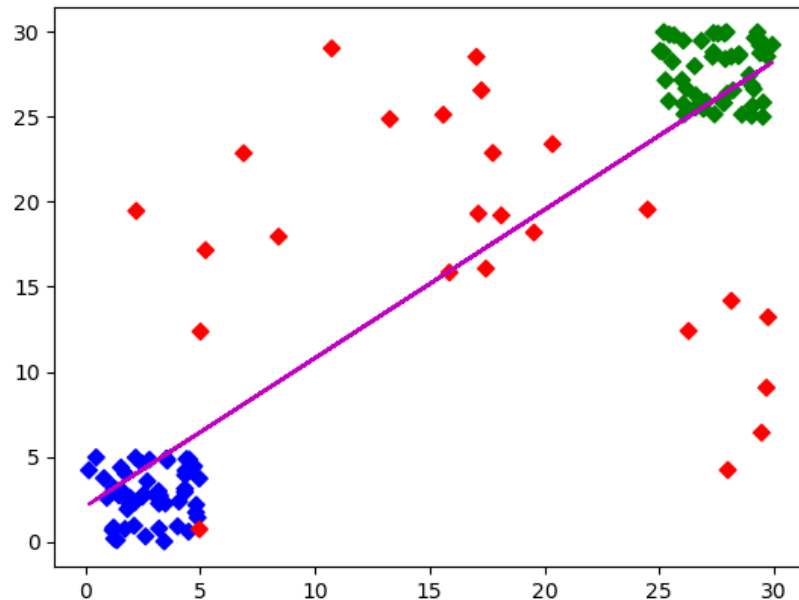
```
In [25]: import numpy as np
import matplotlib.pyplot as plt
x1 = 5 * np.random.rand(50)
x2 = 5 * np.random.rand(50) + 25
x3 = 30 * np.random.rand(25)
x = np.concatenate((x1, x2, x3))
y1 = 5 * np.random.rand(50)
y2 = 5 * np.random.rand(50) + 25
y3 = 30 * np.random.rand(25)
y = np.concatenate((y1, y2, y3))
color_array = ['b'] * 50 + ['g'] * 50 + ['r'] * 25
print(color_array)
plt.scatter(x, y, s=[50], marker='D', c=color_array)
plt.show()
```

Here in this example it uses an array for the colors

Showing correlations

In some cases, you need to know the general direction that your data is taking when looking at a scatterplot. Even if you create a clear depiction of the groups, the actual direction that the data is taking as a whole may not be clear. In this case, you add a trendline to the output.

```
In [6]: import numpy as np
import matplotlib.pyplot as plt
x1 = 5 * np.random.rand(50)
x2 = 5 * np.random.rand(50) + 25
x3 = 30 * np.random.rand(25)
x = np.concatenate((x1, x2, x3))
y1 = 5 * np.random.rand(50)
y2 = 5 * np.random.rand(50) + 25
y3 = 30 * np.random.rand(25)
y = np.concatenate((y1, y2, y3))
color_array = ['b'] * 50 + ['g'] * 50 + ['r'] * 25
print(color_array)
plt.scatter(x, y, s=[30], marker='D', c=color_array)
z = np.polyfit(x, y, 1)
p = np.poly1d(z)
plt.plot(x, p(x), 'm-')
plt.show()
```

[illegible]

- Adding a trendline means calling the NumPy `polyfit()` function with the data, which returns a vector of coefficients, `p`, that minimizes the least squares error. (Least square regression is a method for finding a line that summarizes the relationship between two variables, `x` and `y` in this case, at least within the domain of the explanatory variable `x`. The third `polyfit()` parameter expresses the degree of the polynomial fit.
- The vector output of `polyfit()` is used as input to `poly1d()`, which calculates the actual `y`-axis data points. The call to `plot()` creates the trendline on the scatterplot.

Plotting Time Series

- sometimes you need to view data over a period of time. Only by viewing the data as it changes can you expect to understand the underlying forces that shape it. The following sections describe how to work with data on a time-related basis

Representing time on axes

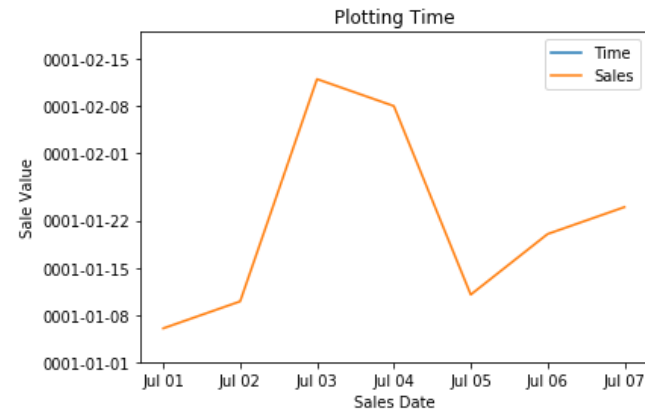
- Many times, you need to present data over time. The data could come in many forms, but generally you have some type of time tick (one unit of time), followed by one or more features that describe what happens during that particular tick. The following example shows a simple set of days and sales on those days for a particular item in whole (integer) amounts.

```
In [55]: import datetime as dt
import pandas as pd
import matplotlib.pyplot as plt
df = pd.DataFrame(columns=('Time', 'Sales'))
start_date = dt.datetime(2015, 7,1)
end_date = dt.datetime(2015, 7,10)
daterange = pd.date_range(start_date, end_date)
print(daterange)
for single_date in daterange:
    row = dict(zip(['Time', 'Sales'],
    [single_date,
    int(50*np.random.rand(1))]))
    row_s = pd.Series(row)
    row_s.name = single_date.strftime('%b %d')
    print(row_s.name)
    df = df.append(row_s)
print(df)
df.loc['Jul 01':'Jul 07', ['Time', 'Sales']].plot()
plt.ylim(1,50)
plt.xlabel('Sales Date')
plt.ylabel('Sale Value')
plt.title('Plotting Time')
plt.show()
```

```
DatetimeIndex(['2015-07-01', '2015-07-02', '2015-07-03', '2015-07-04',
                '2015-07-05', '2015-07-06', '2015-07-07', '2015-07-08',
                '2015-07-09', '2015-07-10'],
              dtype='datetime64[ns]', freq='D')
```

```
Jul 01
Jul 02
Jul 03
Jul 04
Jul 05
Jul 06
Jul 07
Jul 08
Jul 09
Jul 10
```

	Time	Sales
Jul 01	2015-07-01	6
Jul 02	2015-07-02	10
Jul 03	2015-07-03	43
Jul 04	2015-07-04	39
Jul 05	2015-07-05	11
Jul 06	2015-07-06	20
Jul 07	2015-07-07	24
Jul 08	2015-07-08	41
Jul 09	2015-07-09	33
Jul 10	2015-07-10	28



Type *Markdown* and LaTeX: α^2

Visualizing Graphs

A graph is a depiction of data showing the connections between data points using lines. The purpose is to show that some data points relate to other data points, but not all the data points that appear on the graph.

Graphs are a popular data science topic because of their use in social media analysis. When performing social media analysis, you depict and analyze networks of relationships, such as friends or business connections, from social hubs such as Facebook, Google+, Twitter, or LinkedIn.

Use of Graphs for Community Clustering in a SN A community is a group of people/users with some common properties/attributes/ interests/ hobbies/locality/preferences, and/or backgrounds. Community detection in a SN has a range of advantages including target advertising, collaborative filtering, group recommendation, interest-based marketing, information diffusion, personalized advertisement, opinion leader selection, information contagion, and accelerating the information spread.

Graphs can be directed or undirected

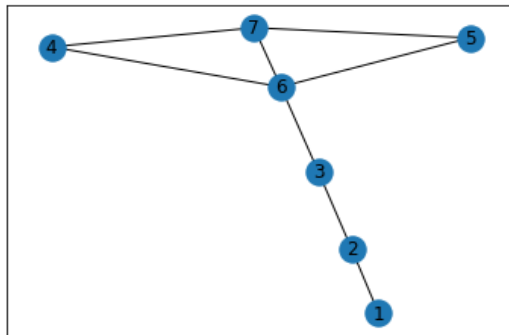
Developing undirected graphs

To create a new undirected graph, the code calls the `Graph()` constructor, which can take a number of input arguments to use as attributes. However, you can build a perfectly usable graph without using attributes, which is what this example does.

The easiest way to add a node is to call `add_node()` with a node number. You can also add a list, dictionary, or `range()` of nodes using `add_nodes_from()`. In fact, you can import nodes from other graphs if you want.

You must define connections(edges) between them. To add a single edge, you call `add_edge()` with the numbers of the nodes that you want to add. As with nodes, you can use `add_edges_from()` to create more than one edge using a list, dictionary, or another graph as input.

```
In [3]: import networkx as nx
import matplotlib.pyplot as plt
G = nx.Graph()
H = nx.Graph()
G.add_node(1)
G.add_nodes_from([2, 3])
G.add_nodes_from(range(4, 7))
H.add_node(7)
G.add_nodes_from(H)
G.add_edge(1, 2)
G.add_edge(1, 1)
G.add_edges_from([(2,3), (3,6), (4,6), (5,6)])
H.add_edges_from([(4,7), (5,7), (6,7)])
G.add_edges_from(H.edges())
nx.draw_networkx(G)
plt.show()
```



Developing directed graphs

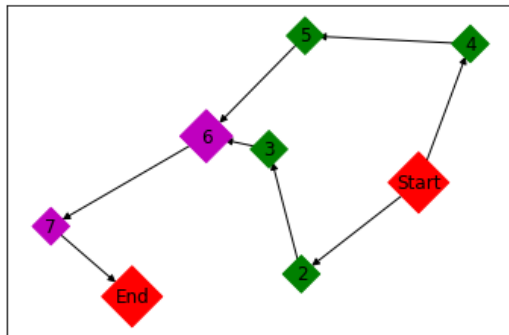
you can create custom labels, use specific colors for certain nodes, or rely on color to help people see the meaning behind your graphs. You can also change edge line weight and use other techniques to mark a specific path between nodes as the better one to choose. The following example shows many (but not nearly all) the ways in which you can dress up a directed graph and make it more interesting:

The example begins by creating a directional graph using the `DiGraph()` constructor. You should note that the NetworkX package also supports `MultiGraph()` and `MultiDiGraph()` graph types.

Adding nodes is much like working with an undirected graph. You can add single nodes using `add_node()` and multiple nodes using `add_nodes_from()`. The `add_path()` call lets you create nodes and edges at the same time. The order of nodes in the call is important. The flow from one node to another is from left to right in the list supplied to the call.

This example adds special node colors, labels, shape (only one shape is used), and sizes to the output. You still call on `draw_networkx()` to perform the task.

```
In [4]: import networkx as nx
import matplotlib.pyplot as plt
G = nx.DiGraph()
H = nx.Graph()
G.add_node(1)
G.add_nodes_from([2, 3])
G.add_nodes_from(range(4, 6))
nx.add_path(G,[6, 7, 8])
G.add_edge(1, 2)
G.add_edges_from([(1,4), (4,5), (2,3), (3,6), (5,6)])
colors = ['r', 'g', 'g', 'g', 'g', 'm', 'm', 'r']
labels = {1:'Start', 2:'2', 3:'3', 4:'4',
          5:'5', 6:'6', 7:'7', 8:'End'}
sizes = [800, 300, 300, 300, 300, 600, 300, 800]
nx.draw_networkx(G, node_color=colors, node_shape='D',
                 with_labels=True, labels=labels,
                 node_size=sizes)
plt.show()
```



```
In [ ]:
```